

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\integration\BaseAbstractUnivariateIntegrator.java

- Specification

module BaseAbstractUnivariateIntegrator

type

```
maxEval: int
minimallterationCount: int
maximallterationCount: int
relativeAccuracy: real
absoluteAccuracy: real
min: real
max: real
ex: set of { None,
             NullArgumentException,
             NotStrictlyPositiveException,
             NumberIsTooLargeException,
             NotANumberException,
             NotFiniteNumberException,
             TooManyEvaluationsException }
```

process integrate(f, min, max, maxEval,
relativeAccuracy, absoluteAccuracy,
minimallterationCount, maximallterationCount: real) result: real, ex:
set of { ... }

Functional Scenario Form

```
f = 0                                && ex = NullArgumentException ||

maxEval <= 0                          && ex =
NotStrictlyPositiveException ||
minimallterationCount <= 0            && ex =
NotStrictlyPositiveException ||
minimallterationCount > maximallterationCount && ex =
NumberIsTooLargeException ||
relativeAccuracy <= 0                && ex = NotStrictlyPositiveException ||

// Domain sanity: numbers must be finite and ordered
isNaN=0                              && ex = NotANumberException ||
isInfinite=0                         && ex = NotFiniteNumberException ||
min > max                            && ex =
NumberIsTooLargeException ||
```

```

// Degenerate interval: integral over zero-length domain
min = max                                && (ex = None and result
= 0.0) ||

```

```

// Runtime evaluation budget exhaustion (generic to base)
evaluations_used > maxEval                && ex = TooManyEvaluationsException ||

```

```

// Normal success (some subclass-specific termination criterion met before budget)
evaluations_used <= maxEval                && ex = None

```

end_process

end_module

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.analysis.solvers.UnivariateSolve
rUtils;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.MaxCountExceededExcept
ion;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;

```

```

import org.apache.commons.math4.legacy.exception.TooManyEvaluationsException;
import org.apache.commons.math4.legacy.core.IntegerSequence;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeException;
import org.apache.commons.math4.legacy.exception.NotANumberException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberException;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsException;
/**
 * Provide a default implementation for several generic functions.
 *
 * @since 1.2
 */
public abstract class BaseAbstractUnivariateIntegrator implements UnivariateIntegrator {
    /** Default absolute accuracy. */
    public static final double DEFAULT_ABSOLUTE_ACCURACY = 1.0e-15;
    /** Default relative accuracy. */
    public static final double DEFAULT_RELATIVE_ACCURACY = 1.0e-6;
    /** Default minimal iteration count. */
    public static final int DEFAULT_MIN_ITERATIONS_COUNT = 3;
    /** Default maximal iteration count. */
    public static final int DEFAULT_MAX_ITERATIONS_COUNT = Integer.MAX_VALUE;
    /** The iteration count. */
    protected IntegerSequence.Incrementor iterations;
    /** Maximum absolute error. */
    private final double absoluteAccuracy;
    /** Maximum relative error. */
    private final double relativeAccuracy;
    /** minimum number of iterations. */
    private final int minimalIterationCount;
    /** maximum number of iterations. */
    private final int maximalIterationCount;
    /** The functions evaluation count. */
    private IntegerSequence.Incrementor evaluations;
    /** Function to integrate. */
    private UnivariateFunction function;
    /** Lower bound for the interval. */

```

```

private double min;
/** Upper bound for the interval. */
private double max;
/**
 * Construct an integrator with given accuracies and iteration counts.
 * <p>
 * The meanings of the various parameters are:
 * <ul>
 * <li>relative accuracy: this is used to stop iterations if the absolute
accuracy can't be achieved due to large values or short mantissa length. If
this should be the primary criterion for convergence rather than a safety
measure, set the absolute accuracy to a ridiculously small value, like
* {@link org.apache.commons.numbers.core.Precision#SAFE_MIN
Precision.SAFE_MIN}.</li>
 * <li>absolute accuracy: The default is usually chosen so that results in the
interval -10...-0.1 and +0.1...+10 can be found with a reasonable accuracy. If
the expected absolute value of your results is of much smaller magnitude, set
this to a smaller value.</li>
 * <li>minimum number of iterations: minimal iteration is needed to avoid false
early convergence, e.g. the sample points happen to be zeroes of the
function. Users can use the default value or choose one that they see as
appropriate.</li>
 * <li>maximum number of iterations: usually a high iteration count indicates
convergence problems. However, the "reasonable value" varies widely for
different algorithms. Users are advised to use the default value supplied by
the algorithm.</li>
 * </ul>
 *
 * @param relativeAccuracy    relative accuracy of the result
 * @param absoluteAccuracy    absolute accuracy of the result
 * @param minimalIterationCount minimum number of iterations

```

```

    * @param maximalIterationCount maximum number of iterations
    * @exception NotStrictlyPositiveException if minimal number of iterations is
    *                                     not strictly positive
    * @exception NumberIsTooSmallException    if maximal number of iterations is
    *                                     lesser than or equal to the minimal
    *                                     number of iterations
    */
    protected BaseAbstractUnivariateIntegrator(final double relativeAccuracy, final double absoluteAccuracy,
        final int minimalIterationCount, final int maximalIterationCount) {
        // accuracy settings
        this.relativeAccuracy = relativeAccuracy;
        this.absoluteAccuracy = absoluteAccuracy;
        // iterations count settings
        if (minimalIterationCount <= 0) {
            throw new NotStrictlyPositiveException(minimalIterationCount);
        }
        if (maximalIterationCount <= minimalIterationCount) {
            throw new NumberIsTooSmallException(maximalIterationCount, minimalIterationCount, false);
        }
        this.minimalIterationCount = minimalIterationCount;
        this.maximalIterationCount = maximalIterationCount;
    }
    /**
     * Construct an integrator with given accuracies.
     *
     * @param relativeAccuracy relative accuracy of the result
     * @param absoluteAccuracy absolute accuracy of the result
     */
    protected BaseAbstractUnivariateIntegrator(final double relativeAccuracy, final double absoluteAccuracy) {
        this(relativeAccuracy, absoluteAccuracy, DEFAULT_MIN_ITERATIONS_COUNT, DEFAULT_MAX_ITERATIONS_COUNT);
    }
    /**
     * Construct an integrator with given iteration counts.
     *
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations

```

```

    * @exception NotStrictlyPositiveException if minimal number of iterations is
    *                                     not strictly positive
    * @exception NumberIsTooSmallException    if maximal number of iterations is
    *                                     lesser than or equal to the minimal
    *                                     number of iterations
    */
    protected BaseAbstractUnivariateIntegrator(final int minimalIterationCount, final int maximalIterationCount) {
        this(DEFAULT_RELATIVE_ACCURACY, DEFAULT_ABSOLUTE_ACCURACY, minimalIterationCount, maximalIterationCount);
    }
    /** {@inheritDoc} */
    @Override
    public double getRelativeAccuracy() {
        return relativeAccuracy;
    }
    /** {@inheritDoc} */
    @Override
    public double getAbsoluteAccuracy() {
        return absoluteAccuracy;
    }
    /** {@inheritDoc} */
    @Override
    public int getMinimalIterationCount() {
        return minimalIterationCount;
    }
    /** {@inheritDoc} */
    @Override
    public int getMaximalIterationCount() {
        return iterations.getMaximalCount();
    }
    /** {@inheritDoc} */
    @Override
    public int getEvaluations() {
        return evaluations.getCount();
    }
    /** {@inheritDoc} */
    @Override
    public int getIterations() {
        return iterations.getCount();
    }
}

```

```

/**
 * @return the lower bound.
 */
protected double getMin() {
    return min;
}

/**
 * @return the upper bound.
 */
protected double getMax() {
    return max;
}

/**
 * Compute the objective function value.
 *
 * @param point Point at which the objective function must be evaluated.
 * @return the objective function value at specified point.
 * @throws TooManyEvaluationsException if the maximal number of function evaluations is exceeded.
 */
protected double computeObjectiveValue(final double point) {
    try {
        evaluations.increment();
    } catch (MaxCountExceededException e) {
        throw new TooManyEvaluationsException(e.getMax());
    }
    return function.value(point);
}

/**
 * Prepare for computation. Subclasses must call this method if they have
 * the
 *
 * {@code integrate} method.
 *
 * @param maxEval Maximum number of evaluations.
 * @param f the integrand function
 * @param lower the min bound for the interval
 * @param upper the upper bound for the interval
 * @throws NullPointerException
 *         if
 *
 * {@code f}

```

```

*
    is
*
    {@code null}.
* @throws org.apache.commons.math4.legacy.exception.MathIllegalArgument
ntException if
*
    {@code min >= max}.
*/
protected void setup(final int maxEval, final UnivariateFunction f, fi
nal double lower, final double upper) {
    // Checks.
    NullArgumentException.check(f);
    UnivariateSolverUtils.verifyInterval(lower, upper);
    // Reset.
    min = lower;
    max = upper;
    function = f;
    iterations = IntegerSequence.Incrementor.create().withMaximalCount(ma
ximalIterationCount);
    evaluations = IntegerSequence.Incrementor.create().withMaximalCount(m
axEval);
}
/** {@inheritDoc} */
@Override
public double integrate(final int maxEval, final UnivariateFunction f,
final double lower, final double upper) {
    // Initialization.
    setup(maxEval, f, lower, upper);
    // Perform computation.
    return doIntegrate();
}
/**
 * Method for implementing actual integration algorithms in derived cl
asses.
 *
 * @return the root.
 * @throws TooManyEvaluationsException if the maximal number of evalua
tions is
 *
 * exceeded.
 * @throws MaxCountExceededException if the maximum iteration count i
s
 *
 * exceeded or the integrator detects
 *
 * convergence problems otherwise

```

```

    */
    protected abstract double doIntegrate();
}
//vt
class BAUVT {
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static void headline(String n) {
        System.out.println("\n==== " + n + " ====");
    }
    static final UnivariateFunction F_SIN = Math::sin;
    static final UnivariateFunction F_LIN = x -> x - 2.0;
    static final UnivariateFunction F_CONST = x -> 5.0;
    static final UnivariateFunction F_ZERO = x -> 0.0;
    public void Vtest() {
        // Base limit in 4.0-beta1: maximalIterationCount ≤ 30
        // So we use 3..30 everywhere.
        SimpsonIntegrator S = new SimpsonIntegrator(1e-12, 1e-12, 3, 30);
        // VT1: null function
        expectThrow("VT1 null function", () -> S.integrate(1000, null, 0.0, 1.0), NullArgumentException.class);
        // VT2: maxEval <= 0
        expectThrow("VT2 maxEval=0", () -> S.integrate(0, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
        // VT3: bad minimalIterationCount <=0
        expectThrow("VT3 minIter<=0 ctor", () -> new SimpsonIntegrator(1e-8, 1e-8, 0, 10),
            NotStrictlyPositiveException.class);
        // VT4: minIter>maxIter
        expectThrow("VT4 minIter>maxIter ctor", () -> new SimpsonIntegrator(1e-8, 1e-8, 10, 5),
            NumberIsTooLargeException.class);
    }
}

```

```

// VT5: relAcc<=0
expectThrow("VT5 relAcc<=0", () -> new SimpsonIntegrator(0.0, 1e-
8, 3, 10), NotStrictlyPositiveException.class);
// VT6: absAcc<=0
expectThrow("VT6 absAcc<=0", () -> new SimpsonIntegrator(1e-
8, 0.0, 3, 10), NotStrictlyPositiveException.class);
SimpsonIntegrator S2 = new SimpsonIntegrator(1e-10, 1e-10, 3, 30);
// VT7: NaN
expectThrow("VT7 NaN min", () -> S2.integrate(1000, F_LIN, Double.NaN
, 1.0), NotANumberException.class);
// VT8: Infinite
expectThrow("VT8 infinite max", () -> S2.integrate(1000, F_LIN, 0.0,
Double.POSITIVE_INFINITY),
NotFiniteNumberException.class);
// VT9: min>max
expectThrow("VT9 min>max", () -> S2.integrate(1000, F_SIN, 2.0, -
1.0), NumberIsTooLargeException.class);
//VT10: degenerate interval (min == max) -> NumberIsTooLargeExceptio
n (beta1
// enforces min<max)
expectThrow("VT10 degenerate interval (min==max)", () -> S2.integrate
(1000, F_CONST, 3.14, 3.14),
NumberIsTooLargeException.class);
// VT11: normal success [0,π]
headline("VT11 ∫ sin(0,π)");
double r11 = S2.integrate(10000, F_SIN, 0.0, Math.PI);
System.out.printf("result=%.17g |err|=%g%n", r11, Math.abs(r11 - 2.0)
);
// VT12: normal success (linear)
headline("VT12 ∫(x-2)[0,4]");
double r12 = S2.integrate(10000, F_LIN, 0.0, 4.0);
System.out.printf("result=%.17g%n", r12);
// VT13: TooManyEvaluationsException
expectThrow("VT13 too few evals", () -> S2.integrate(1, F_SIN, 0.0, 1
0.0), TooManyEvaluationsException.class);
// VT14: zero function
headline("VT14 zero function");
double r14 = S2.integrate(1000, F_ZERO, -10.0, 10.0);
System.out.printf("result=%.17g%n", r14);
// VT15: wide interval stress
headline("VT15 wide interval [-1e3,1e3]");
double r15 = S2.integrate(500000, F_SIN, -1e3, 1e3);
System.out.printf("result=%.17g%n", r15);

```

```

    // VT16: enforce upper-
    bound behavior (>30) -> NumberIsTooLargeException
    expectThrow("VT16 maxIter>30 enforced", () -> new SimpsonIntegrator(1
e-12, 1e-12, 3, 64),
        NumberIsTooLargeException.class);
    // VT17: near-
    degenerate tiny interval -> succeeds, result ~  $f(x) \cdot \epsilon \approx 5 \cdot 1e-12$ 
    headline("VT17 near-degenerate tiny interval succeeds");
    double a = 3.14;
    double b = a + 1e-12; // strictly greater than a
    double r10b = S2.integrate(1000, F_CONST, a, b);
    System.out.printf("VT17 [a=%.14f, b=%.14f] result=%.17g (expected  $\approx 5$ 
e-12)%n", a, b, r10b);
    // VT18: both accuracies <= 0 at ctor -> NotStrictlyPositiveException
    expectThrow("VT18 both accuracies non-
positive", () -> new SimpsonIntegrator(0.0, 0.0, 3, 10),
        NotStrictlyPositiveException.class);
    // VT19: edge case minIter == maxIter (allowed) -> integrate succeeds
    expectThrow("VT19a minIter == maxIter (3==3) forbidden", () -> new Si
mpsonIntegrator(1e-10, 1e-10, 3, 3),
        NumberIsTooSmallException.class);
    // VT20: both endpoints infinite -> NotFiniteNumberException
    expectThrow("VT20 both endpoints infinite",
        () -> S2.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY, Double.PO
SITIVE_INFINITY),
        NotFiniteNumberException.class);
    System.out.println("\n[Done] V-method cases executed.");
}
}
class FBNFT {
    // ----- tiny helper(s) -----
    private static void headline(String n) {
        System.out.println("\n==== " + n + " ====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expe
cted) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s%n", name, expected.getS
impleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.ge
tMessage());

```

```

        System.out.printf("[%s] %s threw expected%n", t.getClass().equals(ex
pected) ? "PASS" : "FAIL", name);
    }
}
// ----- sample functions -----
static final UnivariateFunction F_SIN = Math::sin;
static final UnivariateFunction F_LIN = x -> x - 2.0;
static final UnivariateFunction F_CONST = x -> 5.0;
static final UnivariateFunction F_ZERO = x -> 0.0;
public void Ftest() {
    // Respect 4.0-beta1 constraint: maxIter <= 30 and maxIter > minIter
    SimpsonIntegrator S2 = new SimpsonIntegrator(1e-10, 1e-10, 3, 30);
    // ----- FT01 - FT20 -----
    // FT01: reversed interval (min > max) -> NumberIsTooLargeException
    expectThrow("FT01 reversed interval",
        () -> S2.integrate(1000, F_SIN, 5.005341669616643E307, -
1.1209145429141500E308),
        NumberIsTooLargeException.class);
    // FT02: degenerate interval (min == max) -> NumberIsTooLargeExceptio
n (beta1
    // enforces min<max)
    expectThrow("FT02 degenerate interval (min==max)",
        () -> S2.integrate(1000, F_CONST, -1.7976931348623157E308, -
1.7976931348623157E308),
        NumberIsTooLargeException.class);
    // FT03: NaN min -> NotANumberException
    expectThrow("FT03 NaN min", () -> S2.integrate(5000, F_LIN, Double.Na
N, -9.224617891233441E307),
        NotANumberException.class);
    // FT04: NaN max -> NotANumberException
    expectThrow("FT04 NaN max", () -> S2.integrate(5000, F_SIN, 3.3377819
02114499E306, Double.NaN),
        NotANumberException.class);
    // FT05: both endpoints infinite -> NotFiniteNumberException
    expectThrow("FT05 both endpoints infinite",
        () -> S2.integrate(2000, F_CONST, Double.NEGATIVE_INFINITY, Double.
POSITIVE_INFINITY),
        NotFiniteNumberException.class);
    // FT06: min infinite -> NotFiniteNumberException
    expectThrow("FT06 min is -Infinity",
        () -> S2.integrate(2000, F_SIN, Double.NEGATIVE_INFINITY, 1.2345678
901234567E308),
        NotFiniteNumberException.class);
    // FT07: max infinite -> NotFiniteNumberException

```

```

expectThrow("FT07 max is +Infinity",
    () -> S2.integrate(2000, F_LIN, -
4.778901234567890E307, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// FT08: equal endpoints (huge +) -> NumberIsTooLargeException
expectThrow("FT08 equal endpoints (huge +)",
    () -> S2.integrate(1000, F_ZERO, 9.999999999999999E307, 9.99999999
999999E307),
    NumberIsTooLargeException.class);
// FT09: tiny budget on wide domain -> TooManyEvaluationsException
expectThrow("FT09 budget=1 on wide domain",
    () -> S2.integrate(1, F_SIN, -
1.2345678901234567E307, 1.4567890123456789E307),
    TooManyEvaluationsException.class);
// FT10: normal success on tiny finite interval (constant function)
headline("FT10 tiny finite interval (CONST)");
double ft10 = S2.integrate(10000, F_CONST, -2.000000000000000E-10, -
1.000000000000000E-10);
System.out.printf("FT10 result=%.17g%n", ft10);
// FT11: normal success moderate interval (SIN)
headline("FT11 moderate interval (SIN)");
double ft11 = S2.integrate(20000, F_SIN, -
3.141592653589793E0, 3.141592653589793E0);
System.out.printf("FT11 result=%.17g%n", ft11);
// FT12: zero function on extreme-wide but finite interval
headline("FT12 zero function wide interval");
double ft12 = S2.integrate(100000, F_ZERO, -
7.654321098765432E307, 7.654321098765432E307);
System.out.printf("FT12 result=%.17g%n", ft12);
// FT13: linear on asymmetric large interval (finite)
expectThrow("FT13 linear extreme finite (budget hit)",
    () -> S2.integrate(200000, F_LIN, -
8.765432109876543E305, 1.234567890123457E306),
    TooManyEvaluationsException.class);
// FT14: near-degenerate tiny positive interval (SIN)
headline("FT14 near-degenerate tiny + interval");
double ft14 = S2.integrate(20000, F_SIN, 2.718281828459045E0, 2.71828
1828459046E0);
System.out.printf("FT14 result=%.17g%n", ft14);
// FT15: near-degenerate tiny negative interval (CONST)
headline("FT15 near-degenerate tiny - interval");
double ft15 = S2.integrate(20000, F_CONST, -1.000000000000000E-12, -
9.999999999999999E-13);
System.out.printf("FT15 result=%.17g%n", ft15);

```

```

// FT16: reversed tiny interval -> NumberIsTooLargeException
expectThrow("FT16 reversed tiny interval",
    () -> S2.integrate(5000, F_LIN, 1.000000000000000E-12, -
1.000000000000000E-12),
    NumberIsTooLargeException.class);
// FT17: equal endpoints around zero -> NumberIsTooLargeException
expectThrow("FT17 equal endpoints around zero",
    () -> S2.integrate(5000, F_SIN, -0.000000000000000E0, -
0.000000000000000E0),
    NumberIsTooLargeException.class);
// FT18: +Infinity with finite -> NotFiniteNumberException
expectThrow("FT18 +Infinity with finite",
    () -> S2.integrate(10000, F_CONST, 1.234500000000000E100, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// FT19: -Infinity with finite -> NotFiniteNumberException
expectThrow("FT19 -Infinity with finite",
    () -> S2.integrate(10000, F_SIN, Double.NEGATIVE_INFINITY, -
9.876500000000000E250),
    NotFiniteNumberException.class);
// FT20: budget=1 on moderate interval -> TooManyEvaluationsException
expectThrow("FT20 budget=1 on moderate interval", () -> S2.integrate(
1, F_LIN, -10.0, 10.0),
    TooManyEvaluationsException.class);
System.out.println("\n[Done] FT01-FT20 executed.");
}
}
//z3
class BAUZ3 {
    // --- helpers (keep same style as your V-method harness) ---
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        }
    }
}

```

```

        System.out.printf("[%s] %s threw expected%n", t.getClass().equals(ex
pected) ? "PASS" : "FAIL", name);
    }
}
// --- sample functions (reuse your harness style) ---
static final UnivariateFunction F_SIN = Math::sin;
static final UnivariateFunction F_LIN = x -> x - 2.0;
static final UnivariateFunction F_CONST = x -> 5.0;
static final UnivariateFunction F_ZERO = x -> 0.0;
public void Z3test() {
    // Respect 4.0-beta1 constraint: maxIter <= 30 and maxIter > minIter
    final SimpsonIntegrator S2 = new SimpsonIntegrator(1e-10, 1e-
10, 3, 30);
    // Z301) f = null → ex = NullPointerException
    // ex = 1 f_is_null = true
    expectThrow("Z301 f=null", () -> S2.integrate(1000, null, 0.0, 1.0),
NullPointerException.class);
    // Z302) maxEval <= 0 → ex = NotStrictlyPositiveException
    // ex = 2 maxEval = 0
    expectThrow("Z302 maxEval <= 0 (0)", () -> S2.integrate(0, F_SIN, 0.0
, 1.0),
        NotStrictlyPositiveException.class);
    // Z303) minimalIterationCount <= 0 → ex = NotStrictlyPositiveExcepti
on
    // ex = 2 minimalIterationCount = 0
    expectThrow("Z303 minIter <= 0 (ctor)", () -> new SimpsonIntegrator(1
e-10, 1e-10, 0, 10),
        NotStrictlyPositiveException.class);
    // Z304) minimalIterationCount > maximalIterationCount → ex =
    // NumberIsTooLargeException
    // ex = 3 minIter = 1 maxIter = 0
    expectThrow("Z304 minIter > maxIter (ctor)", () -> new SimpsonIntegra
tor(1e-10, 1e-10, 2, 1),
        NumberIsTooLargeException.class);
    // Z305) relAcc<=0 ∧ absAcc<=0 → ex = NotStrictlyPositiveException
    // ex = 2 relAcc = 0 absAcc = 0
    expectThrow("Z305 relAcc<=0 ∧ absAcc<=0 (ctor)", () -> new SimpsonInt
egrator(0.0, 0.0, 3, 10),
        NotStrictlyPositiveException.class);
    // Z306) isNaN(min) ∨ isNaN(max) → ex = NotANumberException
    // ex = 4 isNaN(min) = true isNaN(max) = false
    expectThrow("Z306 NaN at min", () -> S2.integrate(1000, F_LIN, Double
.NaN, 1.0), NotANumberException.class);

```

```

// Z307) isInfinite(min)  $\vee$  isInfinite(max)  $\rightarrow$  ex = NotFiniteNumberException
// ex = 5 isInf(min) = true isInf(max) = false
expectThrow("Z307 -
Infinity at min", () -> S2.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY, 1.0),
    NotFiniteNumberException.class);
// Z308) min > max  $\rightarrow$  ex = NumberIsTooLargeException
// ex = 3 min = 0 max = -0.5
expectThrow("Z308 min > max", () -> S2.integrate(1000, F_SIN, 0.0, -0.5), NumberIsTooLargeException.class);
// Z309) min = max  $\rightarrow$  (Z3 模板: ex=None, result=0.0)  $\rightarrow$  4.0-beta1
// 实际: NumberIsTooLargeException
// ex = 0 min=max = 0 result = 0 (DIVERGES in beta1)
expectThrow("Z309 min == max (beta1 enforces min<max)", () -> S2.integrate(1000, F_CONST, 3.14, 3.14),
    NumberIsTooLargeException.class);
// Z310) valid_inputs & evaluations_used > maxEval  $\rightarrow$  ex =
// TooManyEvaluationsException
// ex = 6 evals = 2 maxEval = 1
expectThrow("Z310 budget exhausted (maxEval=1)", () -> S2.integrate(1, F_SIN, 0.0, 10.0),
    TooManyEvaluationsException.class);
// Z311) valid_inputs & evals  $\leq$  maxEval  $\rightarrow$  ex = None
// 使用 ZERO 函数 + 小区间 + 小但足够的预算, 确保不超限。
headline("Z311 valid & within budget (ZERO)");
double z311 = S2.integrate(50, F_ZERO, -1e-6, 1e-6);
System.out.printf("Z311 result=%.17g%n", z311);
// Z312) success variant: minIter=maxIter=1  $\rightarrow$  (Z3 模板: ex=None)  $\rightarrow$  4.0-beta1
// 实际: NumberIsTooSmallException
// ex = 0 minIter = 1 maxIter = 1 (DIVERGES in beta1)
expectThrow("Z312 minIter==maxIter==1 (forbidden in beta1)", () -> new SimpsonIntegrator(1e-10, 1e-10, 1, 1),
    NumberIsTooSmallException.class);
// Z313) success variant: absAcc>0, relAcc $\leq$ 0  $\rightarrow$  ex=None
// ex = 0 relAcc = 0 absAcc = 0.5
headline("Z313 absAcc>0, relAcc $\leq$ 0 (success)");
SimpsonIntegrator S_absOnly = new SimpsonIntegrator(0.0, 0.5, 3, 30);
double z313 = S_absOnly.integrate(1000, F_LIN, 0.0, 1.0);
System.out.printf("Z313 result=%.17g%n", z313);
// Z314) success variant: relAcc>0, absAcc $\leq$ 0  $\rightarrow$  ex=None
// ex = 0 relAcc = 0.5 absAcc = 0
headline("Z314 relAcc>0, absAcc $\leq$ 0 (success)");

```

```

SimpsonIntegrator S_relOnly = new SimpsonIntegrator(0.5, 0.0, 3, 30);
double z314 = S_relOnly.integrate(1000, F_LIN, 0.0, 1.0);
System.out.printf("Z314 result=%.17g%n", z314);
// Z315) success variant: mn<mx & 1≤evals≤maxEval → ex=None
// 使用 ZERO 函数 + 极小区间 + 适度预算, 确保在预算内完成。
headline("Z315 mn<mx & evals within budget (ZERO)");
double z315 = S2.integrate(50, F_ZERO, -1e-6, 0.0);
System.out.printf("Z315 result=%.17g%n", z315);
// Z316) edge: evals = maxEval + 1 → ex = TooManyEvaluationsException
// ex = 6 evals = 2 maxEval = 1
expectThrow("Z316 evals > maxEval (edge)", () -> S2.integrate(1, F_SIN, -10.0, 10.0),
    TooManyEvaluationsException.class);
// Z317) within budget (ZERO) → ex = None
// Note: Simpson requires multiple fixed evaluations even for ZERO; use a small
// but adequate budget.
headline("Z317 within budget (ZERO)");
double z317 = S2.integrate(50, F_ZERO, -1e-6, 1e-6);
System.out.printf("Z317 result=%.17g%n", z317);
// Z318) variant: maxEval == 0 → ex = NotStrictlyPositiveException
// ex = 2 maxEval = 0
expectThrow("Z318 maxEval == 0", () -> S2.integrate(0, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
// Z319) variant: isInfinite(max)=true → ex = NotFiniteNumberException
// ex = 5 isInf(min)=false isInf(max)=true
expectThrow("Z319 +Infinity at max", () -> S2.integrate(1000, F_CONST, 0.0, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// Z320) success: tight accuracies & minIter≥2 → ex=None
// ex = 0 relAcc = 0.5 absAcc = 0.5 minIter = 2 maxIter = 2
headline("Z320 tight accuracies & minIter≥2 (success)");
SimpsonIntegrator S_tight = new SimpsonIntegrator(0.5, 0.5, 2, 3);
double z320 = S_tight.integrate(1000, F_SIN, 0.0, Math.PI);
System.out.printf("Z320 result=%.17g%n", z320);
System.out.println("\n[Done] Z301-Z320 executed.");
}
}
class BAUtest {
    public static void main(String[] args) {
        // VT
        BAUVT t1 = new BAUVT();
        t1.Vtest();
    }
}

```

```

// FT
FBNFT t2 = new FBNFT();
// t2.Ftest();
// Z3
BAUZ3 t3 = new BAUZ3();
// t3.Z3test();
}
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ BaseAbstractUnivariateIntegrator.java	60.0 %	6	4	10
> BAUZ3	0.0 %	0	2	2
> FBNFT	0.0 %	0	2	2
▼ BaseAbstractUnivariateIntegrator	100.0 %	4	0	4
BaseAbstractUnivariateIntegrator	100.0 %	4	0	4
BaseAbstractUnivariateIntegrator	100.0 %	0	0	0
computeObjectiveValue(double)	100.0 %	0	0	0
getAbsoluteAccuracy()	100.0 %	0	0	0
getIterations()	100.0 %	0	0	0
getMax()	100.0 %	0	0	0
getMin()	100.0 %	0	0	0
getMinimalIterationCount()	100.0 %	0	0	0
getRelativeAccuracy()	100.0 %	0	0	0
integrate(int, UnivariateFunction)	100.0 %	0	0	0
setup(int, UnivariateFunction, double)	100.0 %	0	0	0
> BAUVT	100.0 %	2	0	2
> BAUtest	100.0 %	0	0	0

3. V-Method testing instruction coverage: 88.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BaseAbstractUnivariateIntegrator.java	35.2 %	487	898	1,385
> BAUZ3	2.8 %	12	412	424
> FBNFT	3.1 %	12	380	392
> BAUVT	79.2 %	336	88	424
> BAUtest	83.3 %	15	3	18
▼ BaseAbstractUnivariateIntegrator	88.2 %	112	15	127
BaseAbstractUnivariateIntegrator	100.0 %	35	0	35
BaseAbstractUnivariateIntegrator	100.0 %	7	0	7
computeObjectiveValue(double)	100.0 %	16	0	16
getAbsoluteAccuracy()	100.0 %	3	0	3
getIterations()	100.0 %	4	0	4
getMax()	100.0 %	3	0	3
getMin()	100.0 %	3	0	3
getMinimalIterationCount()	100.0 %	3	0	3
getRelativeAccuracy()	100.0 %	3	0	3
integrate(int, UnivariateFunction)	100.0 %	9	0	9
setup(int, UnivariateFunction, double)	100.0 %	26	0	26

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ BaseAbstractUnivariateIntegrator.java	40.0 %	4	6	10
> BAUVT	0.0 %	0	2	2
> BAUZ3	0.0 %	0	2	2
▼ BaseAbstractUnivariateIntegrator	50.0 %	2	2	4
BaseAbstractUnivariateIntegrator	50.0 %	2	2	4
BaseAbstractUnivariateIntegrator		0	0	0
computeObjectiveValue(double)		0	0	0
getAbsoluteAccuracy()		0	0	0
getIterations()		0	0	0
getMax()		0	0	0
getMin()		0	0	0
getMinimalIterationCount()		0	0	0
getRelativeAccuracy()		0	0	0
integrate(int, UnivariateFunction)		0	0	0
setup(int, UnivariateFunction, double)		0	0	0
> FBNFT	100.0 %	2	0	2
> BAUtest		0	0	0

5. Fuzz testing instruction coverage: 76.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BaseAbstractUnivariateIntegrator.java	34.7 %	480	905	1,385
> BAUVT	2.8 %	12	412	424
> BAUZ3	2.8 %	12	412	424
▼ BaseAbstractUnivariateIntegrator	76.4 %	97	30	127
BaseAbstractUnivariateIntegrator	57.1 %	20	15	35
BaseAbstractUnivariateIntegrator	100.0 %	7	0	7
computeObjectiveValue(double)	100.0 %	16	0	16
getAbsoluteAccuracy()	100.0 %	3	0	3
getIterations()	100.0 %	4	0	4
getMax()	100.0 %	3	0	3
getMin()	100.0 %	3	0	3
getMinimalIterationCount()	100.0 %	3	0	3
getRelativeAccuracy()	100.0 %	3	0	3
integrate(int, UnivariateFunction)	100.0 %	9	0	9
setup(int, UnivariateFunction, double)	100.0 %	26	0	26
> BAUtest	83.3 %	15	3	18
> FBNFT	87.8 %	344	48	392

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ BaseAbstractUnivariateIntegrator.java	60.0 %	6	4	10
> BAUVT	0.0 %	0	2	2
> FBNFT	0.0 %	0	2	2
▼ BaseAbstractUnivariateIntegrator	100.0 %	4	0	4
BaseAbstractUnivariateIntegrator	100.0 %	4	0	4
BaseAbstractUnivariateIntegrator		0	0	0
computeObjectiveValue(double)		0	0	0
getAbsoluteAccuracy()		0	0	0
getIterations()		0	0	0
getMax()		0	0	0
getMin()		0	0	0
getMinimalIterationCount()		0	0	0
getRelativeAccuracy()		0	0	0
integrate(int, UnivariateFunction)		0	0	0
setup(int, UnivariateFunction, double)		0	0	0
> BAUZ3	100.0 %	2	0	2
> BAUtest		0	0	0

7. Z3 instruction coverage: 88.2%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BaseAbstractUnivariateIntegrator.java		34.7 %	481	904	1,385
> BAUVT		2.8 %	12	412	424
> FBNFT		3.1 %	12	380	392
> BAUZ3		77.8 %	330	94	424
> BAUtest		83.3 %	15	3	18
▼ BaseAbstractUnivariateIntegrator		88.2 %	112	15	127
BaseAbstractUnivariateIntegrator		100.0 %	35	0	35
BaseAbstractUnivariateIntegrator		100.0 %	7	0	7
computeObjectiveValue(double)		100.0 %	16	0	16
getAbsoluteAccuracy()		100.0 %	3	0	3
getIterations()		100.0 %	4	0	4
getMax()		100.0 %	3	0	3
getMin()		100.0 %	3	0	3
getMinimalIterationCount()		100.0 %	3	0	3
getRelativeAccuracy()		100.0 %	3	0	3
integrate(int, UnivariateFunction)		100.0 %	9	0	9
setup(int, UnivariateFunction, double)		100.0 %	26	0	26